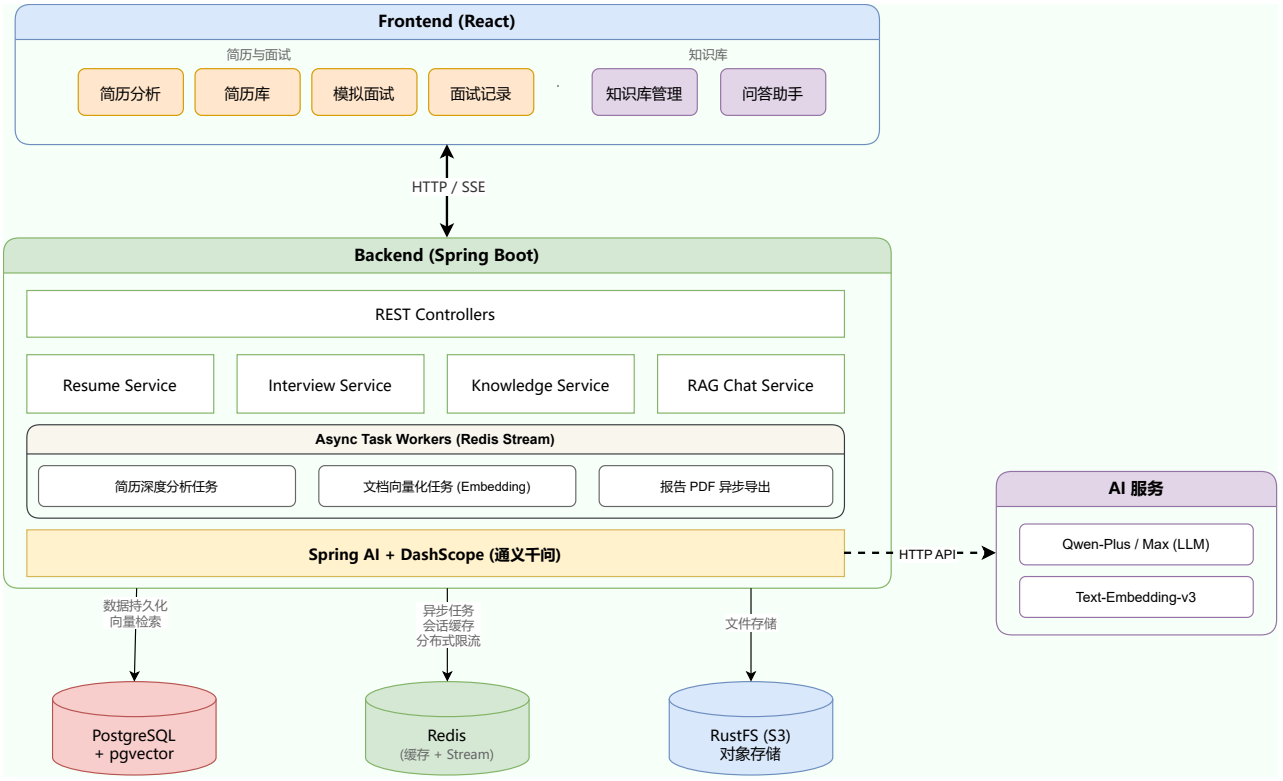
RAG项目架构文档 v1.0

语雀-项目介绍

> 智能 AI 面试官平台 - 系统架构与数据流转分析
>
> 这是一个集成了简历分析、模拟面试和知识库管理的智能面试辅助平台。系统利用大语言模型（LLM）和向量数据库技术，为求职者和 HR 提供智能化的简历评估和面试练习服务。
>
> 基于 Spring Boot 4.0 + Java 21 + Spring AI 2.0 + PostgreSQL + Redis 构建



一、系统整体架构概览

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                                 前端层 (Frontend)                            │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐     │
│  │  简历管理    │  │  模拟面试    │  │  知识库      │  │ RAG问答助手  │       │
│  │  页面        │  │  页面        │  │  管理页面    │  │  页面        │       │
│  └──────┬───────┘  └──────┬───────┘  └──────┬───────┘  └──────┬───────┘     │
│         │                 │                 │                 │             │
│         └─────────────────┴─────────────────┴─────────────────┘             │
│                                   │                                          │
│                              HTTP/REST API                                   │
└──────────────────────────────────┼───────────────────────────────────────────┘
                                   │
                                   ▼

┌──────────────────────────────────────────────────────────────────────────────┐
│                                 后端层 (Backend)                             │
│  ┌──────────────────────────────────────────────────────────────────────┐    │
│  │                     Spring Boot 4.0 应用 (Port: 8080)                 │    │
```

```
│   │  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐      │     │
│   │  │  Resume      │  │  Interview   │  │ Knowledge    │  │   RAG Chat   │      │     │
│   │  │  Controller  │  │  Controller  │  │  Controller  │  │  Controller  │      │     │
│   │  └──────┬───────┘  └──────┬───────┘  └──────┬───────┘  └──────┬───────┘      │     │
│   │         │                 │                 │                 │              │     │
│   │  ┌──────▼───────┐  ┌──────▼───────┐  ┌──────▼───────┐  ┌──────▼───────┐      │     │
│   │  │   Service    │  │   Service    │  │   Service    │  │   Service    │      │     │
│   │  │    Layer     │  │    Layer     │  │    Layer     │  │    Layer     │      │     │
│   │  └──────┬───────┘  └──────┬───────┘  └──────┬───────┘  └──────┬───────┘      │     │
│   │         │                 │                 │                 │              │     │
│   │  ┌──────▼───────┐  ┌──────▼───────┐  ┌──────▼───────┐  ┌──────▼───────┐      │     │
│   │  │ Repository   │  │ Repository   │  │ Repository   │  │ Repository   │      │     │
│   │  │   Layer      │  │   Layer      │  │   Layer      │  │   Layer      │      │     │
│   │  └──────────────┘  └──────────────┘  └──────────────┘  └──────────────┘      │     │
│   └──────────────────────────────────────────────────────────────────────────────┘     │
└──────────────────────────────────────────────────────────────────────────────────────────┘
                                          │
                                          │
                  ┌───────────────────────┼────────────────────────┐
                  │                       │                        │
                  ▼                       ▼                        ▼

┌────────────────┐     ┌──────────────────────────┐     ┌────────────────────┐
│   PostgreSQL   │     │          Redis           │     │   MinIO/RustFS     │
│   (Port: 5432) │     │      (Port: 6379)        │     │  (Port: 9000/9001) │
│                │     │                          │     │                    │
│  • 业务数据存储  │     │  • 面试会话缓存          │     │  • 简历文件存储     │
│  • 向量数据存储  │     │  • 消息队列(Stream)      │     │  • 知识库文档存储   │
│  (pgvector)    │     │  • 限流计数器            │     │  • S3兼容接口       │
└────────┬───────┘     └──────────┬───────────────┘     └──────────┬─────────┘
         │                        │                                 │
         └────────────────────────┼─────────────────────────────────┘
                                  │
                                  ▼

┌──────────────────────────────────────────────────────────────────────────────────────┐
│                                    AI 服务层                                          │
│                                                                                       │
│   阿里云 DashScope (OpenAI 兼容模式)                                                   │
│   • Chat 模型: qwen-plus / qwen-max / qwen-long                                       │
│   • Embedding 模型: text-embedding-v3 (1024维向量)                                    │
│   • 能力: 简历分析、题目生成、答案评估、RAG问答、Query Rewrite                          │
└──────────────────────────────────────────────────────────────────────────────────────┘
```

二、核心功能模块详解

2.1 模块划分

模块	功能描述	核心组件
简历管理模块	简历上传、解析、AI分析、历史管理	ResumeController, ResumeUploadService, ResumeGradingService

模块	功能描述	核心组件
模拟面试模块	创建面试、答题、评估、报告生成	InterviewController, InterviewSessionService, AnswerEvaluationService
知识库模块	文档上传、向量化、RAG检索	KnowledgeBaseController, KnowledgeBaseQueryService, KnowledgeBaseVectorService
RAG问答模块	多轮对话、流式响应、会话管理	RagChatController, RagChatSessionService

---

三、数据流转详细流程

3.1 简历上传与分析流程

```
┌──────────┐     ┌──────────────┐     ┌──────────────┐     ┌──────────────┐
│  前端用户  │──▶│ 选择简历文件  │──▶│  调用上传API  │──▶│  POST /api   │
│          │     │ (PDF/DOCX等) │     │              │     │/resumes/upload│
└──────────┘     └──────────────┘     └──────────────┘     └──────────────┘
                                                                  │
                                                                  ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                        ResumeController.uploadAndAnalyze()                │
└──────────────────────────────────────────────────────────────────────────┘
                                                                  │
                                                                  │
                  ┌──────────────────────────────────────────────┼──────────┐
                  │                                              │          │
                  ▼                                              ▼          ▼
          ┌──────────────┐                          ┌──────────────┐  ┌──────────┐
          │ 1. 文件验证   │                          │ 2. 文件去重检查 │  │3. 解析简历文本 │
          │  - 大小(<10MB) │                          │  - 计算文件Hash │  │  - Apache Tika│
          │  - 格式验证   │                          │  - 查库对比    │  │  - 提取文本   │
          └──────────────┘                          └──────────────┘  └──────────┘
                  │                                        │              │
                  └────────────────────────────────────────┴──────────────┘
                                                │
                                                ▼
                                   ┌──────────────────────────┐
                                   │     4. 存储到 MinIO/RustFS  │
                                   │        (对象存储服务)       │
                                   └──────────────────────────┘
                                                │
                                                ▼
                                   ┌──────────────────────────┐
                                   │   5. 保存简历元数据到 PostgreSQL │
                                   │   状态: ANALYZE_STATUS=PENDING  │
                                   └──────────────────────────┘
                                                │
                                                ▼
                                   ┌──────────────────────────┐
                                   │  6. 发送分析任务到 Redis Stream  │
```

```
                                    |  Stream: resume:analyze:stream   |
                                    └──────────────┬──────────────────┘
                                                   |
                                                   ▼
┌──────────────────────────────────────────────────────────────────────────────┐
|                              立即返回响应 (不等待AI分析)                       |
|  Response: {                                                                 |
|    resume: { id, filename, analyzeStatus: "PENDING" },                       |
|    storage: { fileKey, fileUrl }                                             |
|  }                                                                           |
└──────────────────────────────────────────────────────────────────────────────┘

═══════════════════════════════ 异步处理流程 ═══════════════════════════════

                                    ┌──────────────────────────────┐
                                    |  AnalyzeStreamConsumer (监听)  |
                                    |  消费 resume:analyze:stream     |
                                    └──────────────┬──────────────────┘
                                                   |
                                                   ▼
                                    ┌──────────────────────────────┐
                                    |  7. 更新状态: PROCESSING        |
                                    └──────────────┬──────────────────┘
                                                   |
                                                   ▼
                                    ┌──────────────────────────────┐
                                    |  8. 调用 AI (DashScope)         |
                                    |     - 简历内容分析              |
                                    |     - 生成评分和建议            |
                                    |     - 结构化输出 (JSON)         |
                                    └──────────────┬──────────────────┘
                                                   |
                                                   ▼
                                    ┌──────────────────────────────┐
                                    |  9. 保存分析结果到 PostgreSQL   |
                                    |     更新 ANALYZE_STATUS        |
                                    |     COMPLETED / FAILED         |
                                    └──────────────┬──────────────────┘
                                                   |
                                                   ▼
┌──────────────────────────────────────────────────────────────────────────────┐
|                              前端轮询获取结果                                 |
|  GET /api/resumes/{id}/detail ──▶ 返回最新分析状态和结果                      |
└──────────────────────────────────────────────────────────────────────────────┘
```

状态流转: `PENDING` → `PROCESSING` → `COMPLETED` / `FAILED`

3.2 模拟面试流程

```
┌──────────┐      ┌──────────────────┐      ┌──────────────────┐
│  前端用户  │────▶│  创建面试会话      │────▶│ POST /api/interview│
│          │      │ (选择简历/题目数) │      │     /sessions     │
└──────────┘      └──────────────────┘      └──────────┬──────────┘
                                                        │
                                                        ▼
┌──────────────────────────────────────────────────────────────────────┐
│                      InterviewSessionService.createSession()          │
│                                                                       │
│  1. 检查是否有未完成的会话 (resumeId关联)                               │
│  2. 生成 sessionId (UUID)                                              │
│  3. 调用 AI 生成面试题目 (基于简历内容)                                  │
│  4. 保存会话到 Redis (缓存) + PostgreSQL (持久化)                       │
└──────────────────────────────────────────────────────────────────────┘
                                                        │
                                                        ▼
┌──────────────────────────────────────────────────────────────────────┐
│                              面试进行阶段                              │
│                                                                       │
│  ┌──────────────────────────────────────────────────────────────────┐ │
│  │ GET /api/interview/sessions/{sessionId}/question   ◀── 获取当前题目 │ │
│  │                                                                   │ │
│  │ POST /api/interview/sessions/{sessionId}/answers   ──▶ 提交答案    │ │
│  │      │                                                            │ │
│  │      ▼                                                            │ │
│  │  ┌──────────────────────────────────────┐                         │ │
│  │  │ • 保存答案到 Redis 缓存                 │                         │ │
│  │  │ • 保存答案到 PostgreSQL                 │                         │ │
│  │  │ • currentIndex + 1                     │                         │ │
│  │  │ • 检查是否完成所有题目                   │                         │ │
│  │  └──────────────────────────────────────┘                         │ │
│  │                                                                   │ │
│  │ PUT /api/interview/sessions/{sessionId}/answers    ──▶ 暂存答案    │ │
│  │ (不进入下一题，用于草稿保存)                                          │ │
│  │                                                                   │ │
│  │ POST /api/interview/sessions/{sessionId}/complete  ──▶ 提前交卷    │ │
│  └──────────────────────────────────────────────────────────────────┘ │
└──────────────────────────────────────────────────────────────────────┘

═══════════════════════════ 面试完成/评估 ═══════════════════════════

触发条件: 所有题目回答完成 或 用户提前交卷

                          ┌──────────────────────────┐
                          │ 1. 更新会话状态: COMPLETED  │
                          │    Redis + PostgreSQL     │
                          └──────────┬───────────────┘
                                     │
                                     ▼
                          ┌──────────────────────────┐
```

```
                                    │  2. 发送评估任务到 Redis Stream  │
                                    │  Stream: interview:evaluate:stream│
                                    │  EVALUATE_STATUS=PENDING        │
                                    └─────────────────────────────────┘
                                                    │
                                                    ▼
                                    ┌─────────────────────────────────┐
                                    │  EvaluateStreamConsumer (监听) │
                                    │  分批评估策略 (避免Token溢出)    │
                                    │  - 每批最多8个回答              │
                                    │  - 多轮调用AI进行评估            │
                                    │  - 汇总生成最终报告              │
                                    └─────────────────────────────────┘
                                                    │
                                                    ▼
                                    ┌─────────────────────────────────┐
                                    │  3. 保存评估结果                 │
                                    │     EVALUATE_STATUS=COMPLETED   │
                                    └─────────────────────────────────┘
                                                    │
                                                    ▼
┌──────────────────────────────────────────────────────────────────────────────────┐
│  GET /api/interview/sessions/{sessionId}/report                                  │
│  生成 InterviewReportDTO (分数、评价、建议、统计)                                  │
│                                                                                  │
│  GET /api/interview/sessions/{sessionId}/export                                  │
│  导出 PDF 面试报告                                                               │
└──────────────────────────────────────────────────────────────────────────────────┘
```

会话状态流转: `CREATED` → `IN_PROGRESS` → `COMPLETED` → `EVALUATED`

3.3 知识库上传与向量化流程

```
┌──────────┐      ┌──────────────────┐      ┌──────────────────┐
│  前端用户  │──────▶│  上传知识库文件   │──────▶│ POST /api/knowledgebase│
│          │      │ (PDF/DOCX/MD等)  │      │     /upload      │
└──────────┘      └──────────────────┘      └──────────┬───────┘
                                                        │
                                                        ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│                      KnowledgeBaseUploadService.uploadKnowledgeBase()        │
└──────────────────────────────────────────────────────────────────────────────┘
                                                        │
                    ┌───────────────────────────────────┼──────────────┐
                    │                                   │              │
                    ▼                                   ▼              ▼
          ┌──────────────┐                      ┌──────────────┐  ┌──────────────┐
          │ 1. 文件验证    │                      │ 2. 文件去重检查 │  │3. 解析文档内容 │
          │  - 大小(<50MB) │                      │  - 计算文件Hash │  │  - Apache Tika│
          │  - 格式验证    │                      │  - 查库对比    │  │  - 提取文本   │
          └──────┬───────┘                      └──────┬───────┘  └──────┬───────┘
```

```
                    │                                          │                    │
                    └──────────────────────────────────────────┴────────────────────┘
                                                  │
                                                  ▼
                                     ┌──────────────────────────────┐
                                     │      4. 存储到 MinIO/RustFS     │
                                     └──────────────────────────────┘
                                                  │
                                                  ▼
                                     ┌──────────────────────────────┐
                                     │    5. 保存知识库元数据到 PostgreSQL│
                                     │    状态: VECTOR_STATUS=PENDING   │
                                     └──────────────────────────────┘
                                                  │
                                                  ▼
                                     ┌──────────────────────────────┐
                                     │  6. 发送向量化任务到 Redis Stream│
                                     │  Stream: kb:vectorize:stream     │
                                     └──────────────────────────────┘
                                                  │
                                                  ▼
┌──────────────────────────────────────────────────────────────────────────────────────┐
│                                   立即返回响应                                          │
│  Response: {                                                                           │
│    knowledgeBase: { id, name, vectorStatus: "PENDING", ... },                          │
│    storage: { fileKey, fileUrl },                                                      │
│    duplicate: false                                                                    │
│  }                                                                                     │
└──────────────────────────────────────────────────────────────────────────────────────┘

═══════════════════════════════════ 异步向量化流程 ═══════════════════════════════════

                                     ┌──────────────────────────────┐
                                     │  VectorizeStreamConsumer (监听)│
                                     │  消费 kb:vectorize:stream       │
                                     └──────────────────────────────┘
                                                  │
                                                  ▼
                                     ┌──────────────────────────────┐
                                     │  7. 更新状态: PROCESSING        │
                                     └──────────────────────────────┘
                                                  │
                                                  ▼
                                     ┌──────────────────────────────┐
                                     │  8. 文本分块处理                 │
                                     │     - 按段落/长度分块            │
                                     │     - 准备批量向量化             │
                                     └──────────────────────────────┘
                                                  │
                                                  ▼
                                     ┌──────────────────────────────┐
                                     │  9. 调用 Embedding API          │
                                     │     text-embedding-v3           │
```

```
                                    │     生成 1024维向量                │
                                    └──────────────┬──────────────────┘
                                                   │
                                                   ▼
                                    ┌──────────────────────────────────┐
                                    │  10. 存储向量到 PostgreSQL       │
                                    │      vector_store 表             │
                                    │      (使用 pgvector 扩展)        │
                                    │      - HNSW 索引                 │
                                    │      - COSINE_DISTANCE           │
                                    └──────────────┬──────────────────┘
                                                   │
                                                   ▼
                                    ┌──────────────────────────────────┐
                                    │  11. 更新知识库状态              │
                                    │     VECTOR_STATUS=COMPLETED      │
                                    └──────────────────────────────────┘
```

状态流转: `PENDING` → `PROCESSING` → `COMPLETED` / `FAILED`

3.4 RAG 知识库问答流程

```
┌──────────┐      ┌──────────────────┐      ┌──────────────────────────────────────┐
│  前端用户  │──────▶│    输入问题        │──────▶│  POST /api/knowledgebase/query       │
│          │      │  (选择知识库)      │      │  或 /api/knowledgebase/query/stream │
└──────────┘      └──────────────────┘      └──────────────────────────────────────┘
                                                          │
                                                          ▼
┌──────────────────────────────────────────────────────────────────────────────────┐
│                      KnowledgeBaseQueryService.answerQuestion()                  │
└──────────────────────────────────────────────────────────────────────────────────┘
                                                                  │
                                                                  │
                    ┌──────────────────────────────────────────────┼──────────────┐
                    │                                              │              │
                    ▼                                              ▼              ▼
            ┌────────────────┐                          ┌────────────────┐  ┌────────────────┐
            │ 1. Query处理    │                          │ 2. 向量检索     │  │3. 上下文构建    │
            │  - 归一化       │                         │  - 生成查询向量  │  │  - 合并相关文档 │
            │  - Query Rewrite│                          │  - 相似度搜索    │  │  - 去重排序
│
            │    (可选)       │                         │  - TopK召回     │  │                │
            └────────┬───────┘                          └────────┬───────┘  └────────┬───────┘
                     │                                           │                   │
                     └──────────────────────────────────────────┼───────────────────┘
                                                                │
                                                                ▼
                                    ┌──────────────────────────────────┐
                                    │  4. 构建 RAG Prompt              │
                                    │     System: 角色设定             │
                                    │     User: 上下文 + 问题          │
                                    └──────────────┬──────────────────┘
```

```
                                                  |
                                                  ▼
                                     ┌──────────────────────────────┐
                                     │  5. 调用 LLM (qwen-plus)        │
                                     │     生成回答                     │
                                     └──────────────────────────────┘
                                                  |
                        ┌─────────────────────────┴──────────────────────────┐
                        |                                                    |
                        ▼                                                    ▼
          ┌──────────────────────┐
┌──────────────────────┐
          |     非流式响应         |                                |     流式响应 (SSE)
|
          |                      |                                  |
  |
          |  返回完整回答字符串     |                                |  Flux<String>
|
          |                      |                                  |  - 实时返回Token
|
          └──────────┬───────────┘                                  |  - 探测窗口归一化
|
                     |                                              |  - 避免无信息长文
|
                     ▼
└──────────┬──────────┘
          ┌──────────────────────┐                                          |
          |  保存问答记录          |                                          ▼
          |  (统计/历史)          |
┌──────────────────────┐
          └──────────────────────┘                                  |  流式输出 → 前端
|
                                                                    |  打字机效果展示
|

└──────────────────────┘
```

RAG 优化策略:

- **动态检索参数**: 根据查询长度动态调整 TopK 和相似度阈值
- **Query Rewrite**: 短查询自动扩展，提高召回率
- **多知识库支持**: 支持同时查询多个知识库
- **流式响应**: SSE 实现打字机效果，提升用户体验

3.5 RAG 多轮对话流程

```
┌──────────────────────────────────────────────────────────────────────────────┐
|                              RAG Chat 会话管理                                |
|                                                                              |
|   ┌──────────────────────────────────────────────────────────────────────┐   |
```

```
│  │  会话生命周期                                                          │   │
│  │                                                                        │   │
│  │  POST /api/rag-chat/sessions          ──▶ 创建会话                      │   │
│  │       │                                                                │   │
│  │       ▼                                                                │   │
│  │  ┌──────────────────────────────────────┐                              │   │
│  │  │  • 保存会话到 PostgreSQL                 │                              │   │
│  │  │  • 关联知识库 (可选)                      │                              │   │
│  │  │  • 生成 sessionId                       │                              │   │
│  │  └──────────────────────────────────────┘                              │   │
│  │                                                                        │   │
│  │  GET /api/rag-chat/sessions           ──▶ 获取会话列表                  │   │
│  │  GET /api/rag-chat/sessions/{id}      ──▶ 获取会话详情(含历史消息)       │   │
│  │                                                                        │   │
│  │  PUT /api/rag-chat/sessions/{id}/title    ──▶ 更新会话标题              │   │
│  │  PUT /api/rag-chat/sessions/{id}/pin      ──▶ 置顶/取消置顶             │   │
│  │  PUT /api/rag-chat/sessions/{id}/knowledge-bases ──▶ 切换关联知识库     │   │
│  │                                                                        │   │
│  │  DELETE /api/rag-chat/sessions/{id}    ──▶ 删除会话                     │   │
│  └──────────────────────────────────────────────────────────────────────────┘   │
│                                                                                 │
│  ┌──────────────────────────────────────────────────────────────────────────┐   │
│  │  消息交互 (流式SSE)                                                      │   │
│  │                                                                        │   │
│  │  POST /api/rag-chat/sessions/{id}/messages/stream                      │   │
│  │                                                                        │   │
│  │       │                                                                │   │
│  │       ▼                                                                │   │
│  │  ┌──────────────────────────────────────┐                              │   │
│  │  │  1. 同步保存用户消息到数据库              │                              │   │
│  │  │  2. 创建 AI 消息占位 (状态: pending)      │                              │   │
│  │  │  3. 获取关联知识库列表                    │                              │   │
│  │  └──────────────────────────────────────┘                              │   │
│  │       │                                                                │   │
│  │       ▼                                                                │   │
│  │  ┌──────────────────────────────────────┐                              │   │
│  │  │  4. 调用 RAG 检索 + LLM 生成             │                              │   │
│  │  │     同 3.4 流程                          │                              │   │
│  │  └──────────────────────────────────────┘                              │   │
│  │       │                                                                │   │
│  │       ▼                                                                │   │
│  │  ┌──────────────────────────────────────┐                              │   │
│  │  │  5. 流式返回 (Flux<ServerSentEvent>)     │                              │   │
│  │  │     实时输出 Token                       │                              │   │
│  │  └──────────────────────────────────────┘                              │   │
│  │       │                                                                │   │
│  │       ▼                                                                │   │
│  │  ┌──────────────────────────────────────┐                              │   │
│  │  │  6. 流式完成/错误回调                     │                              │   │
│  │  │     更新 AI 消息内容 (状态: completed)    │                              │   │
│  │  └──────────────────────────────────────┘                              │   │
│  └──────────────────────────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────────────────────┘
```

四、核心服务与中间件

4.1 服务组件清单

服务/组件	技术选型	用途	端口
后端应用	Spring Boot 4.0 + Java 21	核心业务逻辑	8080
前端应用	React 18 + TypeScript + Vite	用户界面	80
PostgreSQL	PostgreSQL 16 + pgvector	业务数据 + 向量存储	5432
Redis	Redis 7	缓存 + 消息队列	6379
MinIO	MinIO	S3兼容对象存储	9000/9001

4.2 Redis 使用场景

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                              Redis 数据结构使用                                │
├──────────────────────────────────────────────────────────────────────────────┤
│                                                                              │
│  1. 面试会话缓存 (Hash)                                                       │
│     Key: interview:session:{sessionId}                                       │
│     Value: { resumeText, questions, currentIndex, status, ... }              │
│     TTL: 24小时                                                              │
│                                                                              │
│  2. 未完成任务索引 (Set)                                                      │
│     Key: interview:unfinished:{resumeId}                                     │
│     Value: sessionId                                                         │
│                                                                              │
│  3. 消息队列 - Redis Stream                                                   │
│     ┌──────────────────────────────────────────────────────────────────┐     │
│     │ Stream Key                    | 用途                              │     │
│     ├──────────────────────────────────────────────────────────────────┤     │
│     │ resume:analyze:stream         | 简历分析任务队列                   │     │
│     │ kb:vectorize:stream           | 知识库向量化任务队列               │     │
│     │ interview:evaluate:stream     | 面试评估任务队列                   │     │
│     └──────────────────────────────────────────────────────────────────┘     │
│                                                                              │
│  4. 限流计数器 (String + Expire)                                              │
│     Key: rate_limit:{dimension}:{identifier}:{timestamp}                     │
│     维度: GLOBAL, IP                                                         │
│                                                                              │
└──────────────────────────────────────────────────────────────────────────────┘
```

4.3 PostgreSQL 数据模型

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                                 核心数据表结构                                 │
├──────────────────────────────────────────────────────────────────────────────┤
│                                                                              │
│  ┌──────────────────┐     ┌──────────────────┐     ┌──────────────────┐     │
│  │   resume         │     │ resume_analysis  │     │ interview_session│     │
│  │   (简历表)       │──▶│   (分析结果表)   │     │  (面试会话表)    │     │
│  ├──────────────────┤     ├──────────────────┤     ├──────────────────┤     │
│  │ id               │     │ id               │     │ id               │     │
│  │ original_filename│     │ resume_id (FK)   │     │ session_id (UK)  │     │
│  │ storage_key      │     │ analysis_result  │     │ resume_id (FK)   │     │
│  │ storage_url      │     │ scores (JSON)    │     │ questions_json   │     │
│  │ resume_text      │     │ suggestions      │     │ status           │     │
│  │ file_hash (UK)   │     │ created_at       │     │ evaluate_status  │     │
│  │ analyze_status   │     └──────────────────┘     └──────────┬───────┘     │
│  │ ...              │                                         │             │
│  └──────────────────┘                                         │             │
│                                                               │             │
│  ┌──────────────────┐     ┌──────────────────┐               │             │
│  │ knowledge_base   │     │  rag_chat_session│               │             │
│  │   (知识库表)     │     │  (RAG会话表)     │               │             │
│  ├──────────────────┤     ├──────────────────┤               │             │
│  │ id               │     │ id               │               │             │
│  │ name             │     │ title            │               │             │
│  │ storage_key      │     │ knowledge_base_ids│              │             │
│  │ file_hash (UK)   │     │ is_pinned        │               │             │
│  │ vector_status    │     │ created_at       │               │             │
│  │ question_count   │     └──────────────────┘               │             │
│  │ ...              │                                         │             │
│  └──────────────────┘                                         ▼             │
│                                                   ┌──────────────────┐     │
│                                                   │interview_answer  │     │
│  ┌──────────────────┐                             │  (面试答案表)    │     │
│  │rag_chat_message  │                             ├──────────────────┤     │
│  │  (RAG消息表)     │                             │ id               │     │
│  ├──────────────────┤                             │ session_id (FK)  │     │
│  │ id               │                             │ question_index   │     │
│  │ session_id (FK)  │                             │ user_answer      │     │
│  │ role             │                             │ ai_evaluation    │     │
│  │ content          │                             │ score            │     │
│  │ status           │                             │ ...              │     │
│  └──────────────────┘                             └──────────────────┘     │
│                                                                            │
│  ┌──────────────────────────────────────────────────────────────────┐      │
│  │ vector_store (Spring AI 自动创建)                                 │      │
│  │  - 向量数据存储 (pgvector)                                        │      │
│  │  - HNSW 索引                                                      │      │
│  │  - metadata 包含 knowledge_base_id 用于过滤                       │      │
│  └──────────────────────────────────────────────────────────────────┘      │
```

```
│                                                                                │
└────────────────────────────────────────────────────────────────────────────────┘
```

五、关键技术实现细节

5.1 异步任务处理机制

```java
// 生产者: 发送任务到 Redis Stream
@Component
public class AnalyzeStreamProducer {
    public void sendAnalyzeTask(Long resumeId, String content, String resumeType, String
targetPosition) {
        Map<String, String> message = new HashMap<>();
        message.put("resumeId", resumeId.toString());
        message.put("content", content);
        message.put("resumeType", resumeType);
        message.put("targetPosition", targetPosition);

        redisService.streamAdd(RESUME_ANALYZE_STREAM_KEY, message, STREAM_MAX_LEN);
    }
}

// 消费者: 监听并处理任务
@Component
public class AnalyzeStreamConsumer extends AbstractStreamConsumer<AnalyzePayload> {
    @Override
    protected void processBusiness(AnalyzePayload payload) {
        // 1. 更新状态为 PROCESSING
        markProcessing(payload);

        // 2. 调用 AI 进行分析
        ResumeAnalysisResponse analysis = gradingService.analyzeResume(
            payload.content(), payload.resumeType(), payload.targetPosition()
        );

        // 3. 保存结果
        persistenceService.saveAnalysis(resume, analysis);

        // 4. 更新状态为 COMPLETED
        markCompleted(payload);
    }
}
```

5.2 RAG 检索增强流程

```
用户问题
    │
    ▼
┌──────────────────┐
│ Query Rewrite    │ ◀── 短查询自动扩展 (可选)
```

```
│ (Query优化)      │
└──────────────────┘
         │
         ▼
┌──────────────────┐
│  Embedding       │ ◀── 调用 text-embedding-v3
│  向量化          │
└──────────────────┘
         │
         ▼
┌──────────────────┐
│ 相似度搜索       │ ◀── pgvector COSINE_DISTANCE
│ (TopK召回)       │     动态参数: 短查询 TopK=20, 长查询 TopK=8
└──────────────────┘
         │
         ▼
┌──────────────────┐
│ 上下文构建       │ ◀── 合并相关文档片段
│ (Context构建)    │
└──────────────────┘
         │
         ▼
┌──────────────────┐
│  Prompt组装      │ ◀── System + User(Context + Question)
│                  │
└──────────────────┘
         │
         ▼
┌──────────────────┐
│ LLM生成回答      │ ◀── qwen-plus 流式/非流式
│                  │
└──────────────────┘
```

5.3 面试评估分批策略

```
面试回答评估 (避免Token溢出)
│
├─ 收集所有回答 (N个)
│
├─ 计算分批: batchSize = 8
│   批次数量 = ceil(N / 8)
│
├─ 循环每批:
│   ├─ 构建 Prompt (包含该批8个回答)
│   ├─ 调用 AI 评估
│   └─ 收集评估结果
│
├─ 汇总所有批次结果
│
└─ 生成最终报告 (含综合评分、建议、统计)
```

六、系统交互时序图

6.1 简历上传完整时序

```
用户      前端      ResumeController  ResumeUploadService  FileStorageService  PostgreSQL
RedisStream  AnalyzeConsumer  AI服务
 |        |              |                   |                   |            |
|               |        |
 |──上传──▶|              |                   |                   |            |
|               |        |
 |        |──────────────POST
/api/resumes/upload──────────────────────────────────────────────────────────────────▶
 |        |              |                   |                   |            |
|               |        |
 |        |              |──────────────────▶                    |            |
|               |        |
 |        |              |                   |                   |            |
|               |        |
 |        |              |                   |──1. 验证文件──────▶|            |
|               |        |
 |        |              |                   |                   |            |
|               |        |
 |        |              |                   |──2. 检查重复
──────────────────────────────────────────────────────────────▶
 |        |              |                   |                   |            |
|               |        |
 |        |              |                   |──3. 解析文本──────▶|            |
|               |        |
 |        |              |                   |                   |            |
|               |        |
 |        |              |                   |──4. 上传MinIO──────────────────▶|
   |               |        |
 |        |              |                   |                   |            |
|               |        |
 |        |              |                   |──5. 保存元数据──────────────────▶|
|               |        |
 |        |              |                   |                   |            |
|               |        |
 |        |              |                   |──6. 发送Stream任务
──────────────────────────────────────▶|        |
 |        |              |                   |                   |            |
|               |        |
 |        |◀──────────────Response
(PENDING)──────────────────────────────────────────────────────────────────────────|
 |◀───────|              |                   |                   |            |
|               |        |
 |        |              |                   |                   |            |
|               |        |
 |        |              |                   |                   |            |
|◀──────────────|        |
 |        |              |                   |                   |            |
|               |        |
```

```
 |        |              |                  |                  |            |
|──7. 更新状态 PROCESSING
 |        |              |                  |                  |            |
|──────────────▶|        |
 |        |              |                  |                  |            |
|               |        |
 |        |              |                  |                  |            |
|──8. 调用AI分析──────────────────▶
 |        |              |                  |                  |            |
|               |        |
 |        |              |                  |                  |            |
|◀──────────────|        |
 |        |              |                  |                  |            |
|               |        |
 |        |              |                  |                  |            |
|──9. 保存结果──────────────────▶
 |        |              |                  |                  |            |
|               |        |
 |        |              |                  |                  |            |
|──10. 更新状态 COMPLETED
 |        |              |                  |                  |            |
|──────────────▶|        |
 |        |              |                  |                  |            |
|               |        |
 |        |◀──────────────GET /api/resumes/{id}/detail (轮
询)──────────────────────────────────────────────────────────▶
 |◀───────|              |                  |                  |            |
|               |        |
```

七、部署架构

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                              Docker Compose 部署                              │
├──────────────────────────────────────────────────────────────────────────────┤
│                                                                              │
│   ┌──────────────┐    ┌──────────────┐    ┌──────────────┐    ┌──────────────┐          │
│   │   postgres   │    │    redis     │    │    minio     │    │ createbuckets│          │
│   │  (数据库)    │    │  (缓存/MQ)   │    │  (对象存储)  │    │ (初始化任务) │          │
│   │   :5432      │    │   :6379      │    │  :9000/9001  │    │   (一次性)   │          │
│   └──────┬───────┘    └──────┬───────┘    └──────┬───────┘    └──────┬───────┘          │
│          │                   │                   │                   │                  │
│          └───────────────────┴───────────────────┴───────────────────┘                  │
│                                      │                                                  │
│                                      ▼                                                  │
│                              ┌──────────────┐                                           │
│                              │     app      │                                           │
│                              │  (Spring Boot)                                           │
│                              │   :8080      │                                           │
│                              └──────┬───────┘                                           │
│                                     │                                                   │
│                                     ▼                                                   │
```

```
│                              ┌──────────────┐                                  │
│                              │   frontend   │                                  │
│                              │   (React)    │                                  │
│                              │    :80       │                                  │
│                              └──────────────┘                                  │
│                                                                               │
│  数据持久化:                                                                   │
│  • postgres_data ──▶ PostgreSQL 数据                                          │
│  • redis_data ───▶ Redis 数据                                                 │
│  • minio_data ───▶ MinIO 对象存储                                             │
│                                                                               │
└───────────────────────────────────────────────────────────────────────────────┘
```

八、技术亮点总结

8.1 架构设计亮点

亮点	说明
异步解耦	Redis Stream 实现异步任务，避免AI调用阻塞用户请求
缓存优先	面试会话使用 Redis 缓存，支持断点续面
向量检索	pgvector 实现 RAG，无需额外向量数据库
流式响应	SSE 实现打字机效果，提升交互体验
分批处理	面试评估分批进行，规避 Token 溢出
虚拟线程	Java 21 虚拟线程，提升 I/O 密集型并发能力

8.2 可靠性保障

机制	实现方式
重复检测	文件 Hash 去重，避免重复处理
任务重试	Stream Consumer 自动重试 (最多3次)
状态管理	PENDING → PROCESSING → COMPLETED/FAILED
限流保护	注解式限流 (@RateLimit)，支持多维度
优雅降级	AI 服务异常时返回友好错误提示

文档生成时间: 2026-03-22
项目版本: 基于 interview-guide 开源项目分析